

CSCS Frontend Flow — Call These Endpoints in This Order

This is the frontend guide for the normal successful CSCS process. The frontend must use the status returned by the API to decide what happens next. Never advance the status in the browser yourself.

The complete successful flow

MAKER

1. POST `/api/cscs/import`
Save `data.batch_id`. Expected status: `PROCESSING`
2. GET `/api/cscs/uploads/{batchId}`
Poll until `workflow_status` is `DRAFT_REVIEW`
3. GET `/api/cscs/uploads/{batchId}/preview`
GET `/api/cscs/uploads/{batchId}/exceptions`
4. If exceptions need action, repeat for each one:
POST `/api/cscs/uploads/{batchId}/exceptions/{exceptionId}/resolve`
5. POST `/api/cscs/uploads/{batchId}/reconcile`
Expected status: `RECONCILED`
6. POST `/api/cscs/uploads/{batchId}/submit`
Expected status: `PENDING_APPROVAL`

CHECKER — a different logged-in user

7. GET `/api/cscs/uploads/{batchId}/preview`
GET `/api/cscs/uploads/{batchId}/reconciliation`
8. POST `/api/cscs/uploads/{batchId}/approve`
Final approval status: `APPROVED_AWAITING_POST`
If still `PENDING_APPROVAL`, another checker must approve

POSTER — cannot be the maker

9. POST `/api/cscs/uploads/{batchId}/post`
Expected status: `POSTING_QUEUED`
10. GET `/api/cscs/uploads/{batchId}/posting-status`
Poll until status is `POSTED`

Successful status order:

PROCESSING

- > DRAFT_REVIEW
- > RECONCILED
- > PENDING_APPROVAL
- > APPROVED_AWAITING_POST
- > POSTING_QUEUED
- > POSTING
- > POSTED

1. Common request setup

All endpoints use this base URL:

```
{BASE_URL}/api
```

Send these headers on every request:

```
Accept: application/json
Authorization: Bearer <token>
```

For JSON requests, also send `Content-Type: application/json`.

Do not manually set `Content-Type` when uploading browser `FormData`; the browser must create the multipart boundary.

There are two response shapes to know.

Action endpoints such as upload, reconcile, submit, approve, and post return:

```
{
  "success": true,
  "message": "...",
  "data": {
    "batch_id": 101,
    "status": "RECONCILED"
  }
}
```

The batch-details endpoint returns:

```
{
  "data": {
    "id": 101,
    "workflow_status": "RECONCILED",
    "allowed_actions": ["view", "submit"]
  }
}
```

Therefore:

- After a `POST`, read `response.data.data.status`.
- After `GET /uploads/{batchId}`, read `response.data.data.workflow_status`.

2. Maker uploads the files

```
POST /api/cscs/import
```

Permission: `cscs.upload`

Send `multipart/form-data`:

Field	Required	Value
<code>register_id</code>	Yes	Existing register ID
<code>files[]</code>	Yes	One or two <code>.txt</code> / <code>.csv</code> files
<code>description</code>	No	Maximum 500 characters
<code>business_reference</code>	No	Maximum 100 characters

Rules:

- A movement file is required.
- If sending two files, send one movement file and one master file.
- Maximum file size is 20 MB per file by default.
- Append each file using the exact field name `files[]`.

```
async function uploadCscs({ registerId, files, description, businessReference }) {
  const form = new FormData();
  form.append('register_id', String(registerId));
  if (description) form.append('description', description);
  if (businessReference) form.append('business_reference', businessReference);
  files.forEach(file => form.append('files[]', file));

  const response = await api.post('/api/cscs/import', form);
  return response.data.data;
}
```

Success is HTTP `202 Accepted`:

```
{
  "success": true,
  "message": "CSCS files accepted for processing",
  "data": {
    "batch_id": 101,
    "status": "PROCESSING",
    "summary": {
      "processing_stage": "STAGED",
      "processing_percent": 0
    }
  }
}
```

After success:

1. Save `data.batch_id`.
2. Open the batch page.
3. Start Step 3 polling.
4. Display “Processing CSCS files,” not “Posted successfully.”

Uploading does not change live holdings.

3. Wait for processing

```
GET /api/cscs/uploads/{batchId}
```

Permission: `cscs.view`

Poll every three seconds while `data.workflow_status` is `PROCESSING`:

```
async function waitForCscsProcessing(batchId, signal) {
  while (!signal.aborted) {
    const response = await api.get(`/api/cscs/uploads/${batchId}`, { signal });
    const batch = response.data.data;

    if (batch.workflow_status !== 'PROCESSING') return batch;
    await new Promise(resolve => setTimeout(resolve, 3000));
  }
}
```

Status	What the frontend does next
<code>PROCESSING</code>	Keep polling and show <code>summary.processing_percent</code>
<code>DRAFT_REVIEW</code>	Stop polling and open the review screen
<code>PROCESSING_FAILED</code>	Stop, show <code>failure_reason</code> , and require a corrected new upload

Progress is based on actual backend work and never moves backwards:

summary.processing_stage	Percentage	Suggested label
STAGED	0	Upload accepted
PARSING	1–80	Reading CSCS rows
VALIDATING	82	Starting validation
VALIDATING_ROWS	82–88	Validating movement rows
VALIDATING_TRANSACTIONS	88–95	Validating transaction pairs
CALCULATING_EFFECTS	95–99	Calculating holding effects
FINALIZING	99	Finalizing batch
READY	100	Ready for review
FAILED	Last completed percentage	Processing stopped; show failure_reason

During parsing, the response also contains `summary.source_rows_processed` and `summary.source_rows_total`. The frontend may display, for example, “Reading row 4,250 of 10,000.”

Cancel polling when the page unmounts.

4. Maker reviews the batch

When status becomes `DRAFT_REVIEW`, call these in parallel:

```
GET /api/cscs/uploads/{batchId}/preview
GET /api/cscs/uploads/{batchId}/exceptions?per_page=100
```

Use preview to show the batch summary, debit/credit totals, current/proposed holdings in `account_effects`, mappings, and risk flags.

The exceptions endpoint is paginated. Its exception rows are in `response.data.data`.

Load transaction pairs for a Transactions tab with:

```
GET /api/cscs/uploads/{batchId}/transactions?per_page=100
```

Do not display Post here. The batch is not reconciled or approved.

5. Maker resolves exceptions when present

Skip this step if no exceptions require a decision.

For each exception, call:

```
POST /api/cscs/uploads/{batchId}/exceptions/{exceptionId}/resolve
Content-Type: application/json
```

Permission: `cscs.reconcile`

`exceptionId` is the row `id` returned by the exceptions endpoint.

Map to an existing register account:

```
{
  "resolution_type": "MAP_ACCOUNT",
  "register_account_id": 456,
  "reason": "Matched to the verified shareholder register account"
}
```

The selected account must belong to the batch register.

Exclude under an approved rule:

```
{
  "resolution_type": "RULE_EXCLUDED",
  "reason": "Excluded under the approved documented reconciliation rule"
}
```

Confirm a previously posted movement:

```
{
  "resolution_type": "CONFIRM_REPLAY",
  "reason": "Confirmed as an already posted CSCS movement"
}
```

The API rejects replay confirmation when no matching posted movement exists. The `reason` is always required and must be 10–1,000 characters.

After each resolution, show the success message and reload both `/exceptions` and `/preview`.

Resolving exceptions is not the same as reconciliation. Always continue to Step 6.

6. Maker reconciles

```
POST /api/cscs/uploads/{batchId}/reconcile
Content-Type: application/json
```

Permission: `cscs.reconcile`

```
{
  "comment": "All exceptions, transaction pairs and holding effects reviewed"
}
```

`comment` is optional, maximum 1,000 characters.

Expected response status:

```
RECONCILED
```

After success, reload `GET /api/cscs/uploads/{batchId}` and show Submit only when `allowed_actions` contains `submit`.

On HTTP `422`, display the validation errors, reload preview and exceptions, and remain on review. This usually means a blocking exception remains.

Use `/reconcile` in new frontend code. `/revalidate` is only an alias.

7. Maker submits for approval

```
POST /api/cscs/uploads/{batchId}/submit
Content-Type: application/json
```

Permission: `cscs.submit`

Only the original uploader can submit, and status must be `RECONCILED`.

```
{
  "comment": "Submitting the reconciled CSCS batch for independent review"
}
```

Expected status:

```
PENDING_APPROVAL
```

After success, reload the batch, make the maker view read-only, and show “Waiting for checker approval.” Never show Approve to the maker.

8. Checker reviews

The checker must be a different logged-in user from the maker.

Call:

```
GET /api/cscs/uploads/{batchId}
GET /api/cscs/uploads/{batchId}/preview
GET /api/cscs/uploads/{batchId}/reconciliation
GET /api/cscs/uploads/{batchId}/transactions?per_page=100
```

Before enabling Approve, verify that `workflow_status` is `PENDING_APPROVAL` and `allowed_actions` contains `approve`.

The backend still enforces all permissions and maker-checker restrictions.

9. Checker approves

```
POST /api/cscs/uploads/{batchId}/approve
Content-Type: application/json
```

Permission: `cscs.approve`

```
{
  "comment": "Source files, balances and proposed holdings verified"
}
```

Read `response.data.data.status`:

Returned status	Meaning	Next action
<code>APPROVED_AWAITING_POST</code>	Final approval completed	Continue to posting
<code>PENDING_APPROVAL</code>	Another approval is required	Reload and wait for another eligible checker

For multiple approval steps:

- Display `current_approval_step` and `required_approval_steps`.
- Call the same `/approve` endpoint for every step.
- A different eligible user must approve each step.
- One user cannot approve multiple steps for the same revision.

Approval does not change live holdings.

10. Poster releases the batch

Call only at `APPROVED_AWAITING_POST`:

```
POST /api/cscs/uploads/{batchId}/post
Content-Type: application/json
```

Permission: `cscs.post`


```
{
  "comment": "Approved CSCS batch released to live holdings"
}
```

The poster cannot be the maker. The active policy may also require the poster to be different from the final checker.

Success is HTTP `202 Accepted` with status `POSTING_QUEUED`. This means queued, not completed. Disable the Post button when the request begins and immediately start polling.

11. Poll until posting finishes

```
GET /api/cscs/uploads/{batchId}/posting-status
```

Permission: `cscs.view`

Poll every three seconds during `POSTING_QUEUED` or `POSTING`:

```
async function waitForCscsPosting(batchId, signal) {
  const working = new Set(['POSTING_QUEUED', 'POSTING']);

  while (!signal.aborted) {
    const response = await api.get(
      `/api/cscs/uploads/${batchId}/posting-status`,
      { signal }
    );
    const result = response.data.data;

    if (!working.has(result.status)) return result;
    await new Promise(resolve => setTimeout(resolve, 3000));
  }
}
```

Final success example:

```
{
  "data": {
    "batch_id": 101,
    "status": "POSTED",
    "posted_at": "2026-08-20T10:00:04.000000Z",
    "failure_reason": null,
    "posted_rows": 200
  }
}
```

Final status	What to do
POSTED	Show success and reload preview, transactions, and affected holdings
POSTING_FAILED	Show <code>failure_reason</code> ; offer an authorized manual retry
STALE	Return the batch to maker reconciliation and repeat approval

Never automatically retry financial posting.

Manual retry after `POSTING_FAILED`:

```
POST /api/cscs/uploads/{batchId}/retry-posting
Content-Type: application/json
```

```
{
  "comment": "Retry approved after reviewing the posting failure"
}
```

After retry success, resume `/posting-status` polling.

12. Frontend status router

After every successful action, reload `GET /api/cscs/uploads/{batchId}` and route the UI using `workflow_status`:

```
function getCscsScreen(status) {
  switch (status) {
    case 'PROCESSING': return 'processing';
    case 'DRAFT_REVIEW':
    case 'RECONCILED': return 'maker-review';
    case 'PENDING_APPROVAL': return 'checker-review';
    case 'QUERY_RAISED': return 'maker-query';
    case 'APPROVED_AWAITING_POST': return 'posting-release';
    case 'POSTING_QUEUED':
    case 'POSTING': return 'posting-progress';
    case 'POSTED': return 'success';
    case 'POSTING_FAILED': return 'posting-failed';
    case 'STALE': return 'maker-review';
    case 'PROCESSING_FAILED':
    case 'REJECTED':
    case 'CANCELLED': return 'finished-without-posting';
    default: return 'batch-details';
  }
}
```

Use `allowed_actions` to decide which buttons to render:

Action	Button	Endpoint
resolve	Resolve exception	POST /exceptions/{exceptionId}/resolve
revalidate	Reconcile	POST /reconcile
submit	Submit	POST /submit
query	Raise query	POST /query
approve	Approve	POST /approve
reject	Reject	POST /reject
post	Post/retry	POST /post or /retry-posting
cancel	Cancel	POST /cancel

The action is named `revalidate` in `allowed_actions`, but its recommended endpoint is `/reconcile`.

13. Checker query or rejection

These are not part of the happy path, but the UI needs them.

Checker raises a query:

```
POST /api/cscs/uploads/{batchId}/query
```

```
{
  "comment": "Please confirm the selected debit account",
  "transaction_numbers": ["2606160005615022"],
  "row_ids": [501]
}
```

Status becomes `QUERY_RAISED`. The maker responds:

```
POST /api/cscs/uploads/{batchId}/respond-to-query
```

```
{
  "comment": "The account mapping and supporting record have been confirmed"
}
```

The batch returns to `DRAFT_REVIEW`. Repeat review/resolve → reconcile → submit → approve → post.

Checker rejection:

```
POST /api/cscs/uploads/{batchId}/reject
```

```
{
  "comment": "The source advice does not match the approved instruction"
}
```

REJECTED is read-only. Start a corrected upload as a new batch.

14. Error handling

HTTP status	Frontend behavior
401	Refresh authentication or return to login
403	Show API message; permission or maker-checker rule failed
404	Show “Batch not found” and return to list
422	Show the errors fields; do not advance the flow
429	Prevent repeated clicks and ask the user to retry later
500	Show the safe API message and preserve current page data

For every mutation button:

1. Disable it while the request runs.
2. Never update workflow state optimistically.
3. On success, show the API **message** and reload the batch.
4. On failure, remain on the current step and show the API error.

15. Users and permissions needed

Task	Permission
View	cscs.view
Upload	cscs.upload
Resolve/reconcile	cscs.reconcile
Submit	cscs.submit
Query/review	cscs.review
Approve/reject	cscs.approve
Post/retry	cscs.post

Use at least two accounts in testing:

- Maker: upload, resolve, reconcile, submit.

- Checker: review and approve.

Use a third Poster when policy prevents the checker from posting. The maker can never approve or post their own batch.

16. Completion checklist

- Upload uses repeated `files[]` fields and saves `batch_id`.
- Processing polling stops at `DRAFT_REVIEW` or `PROCESSING_FAILED`.
- Preview and exceptions load after processing.
- Maker can resolve exceptions, reconcile, then submit.
- Maker cannot approve or post.
- Checker can approve only at `PENDING_APPROVAL`.
- Multiple approvals stay `PENDING_APPROVAL` until the final step.
- Poster posts only at `APPROVED_AWAITING_POST`.
- Polling continues through `POSTING_QUEUED` and `POSTING`.
- Success is displayed only after `/posting-status` returns `POSTED`.
- Mutation buttons prevent double clicks.
- The batch reloads after every mutation.